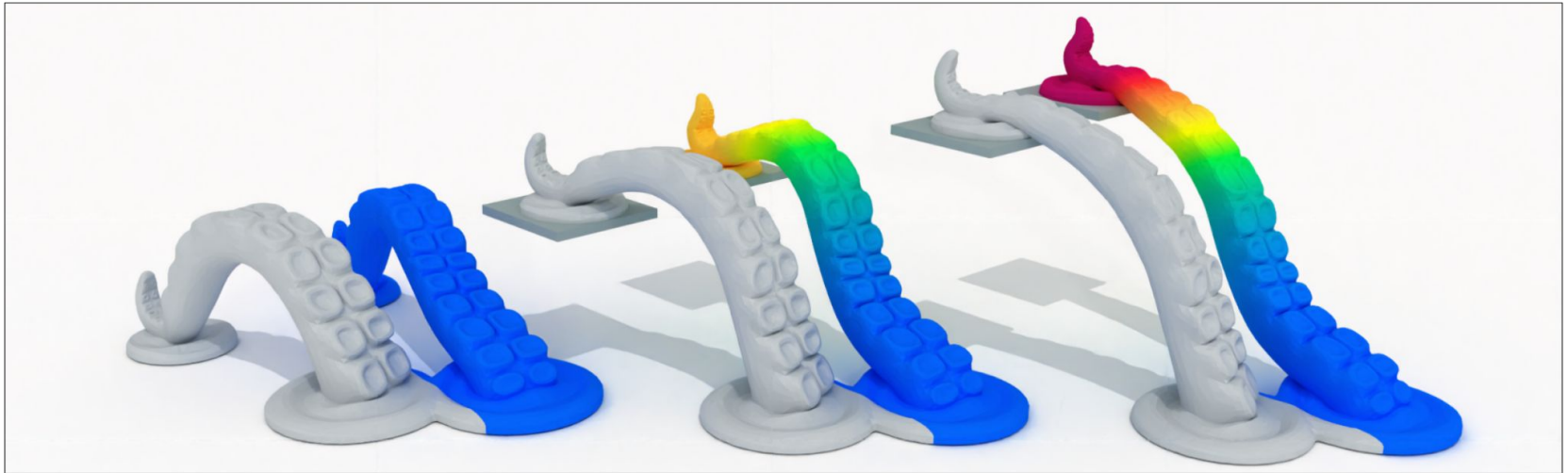


Going further with Vertex Bloc Descent

Bastien Saillant, Florence Zara, Fabrice Jaillet, Guillaume Damiand

INSA Lyon, CNRS, Université Claude Bernard Lyon 1, LIRIS, UMR5205, 69621 Villeurbanne, France



Presenter : Bastien Saillant

Summary

Problematic and Contributions

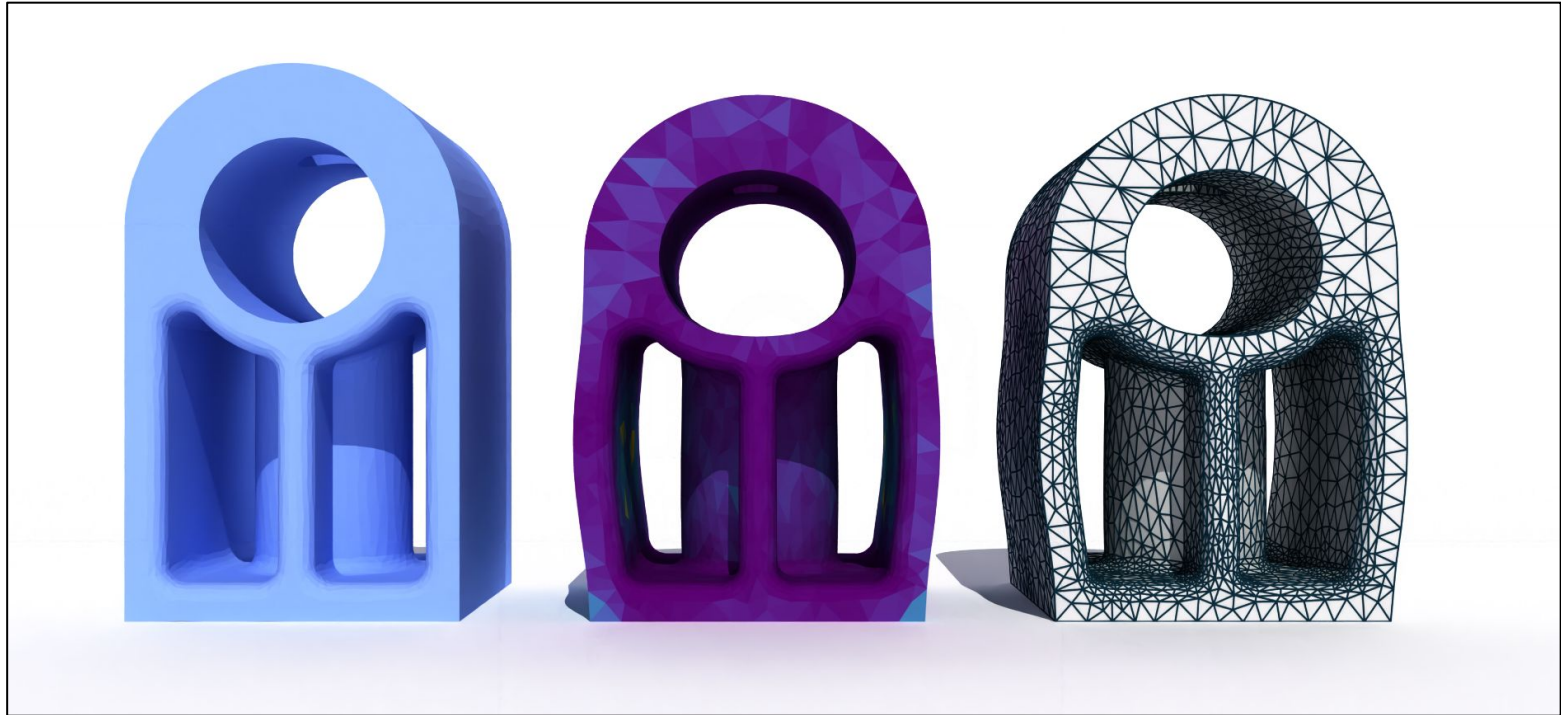
Introduction

Method

Results

Conclusion and Limitations

Context



Deformable Object Simulation

Finite element method

Real-Time/Fast Simulation

Related work

Vertex Block Descent [Chen 2024]

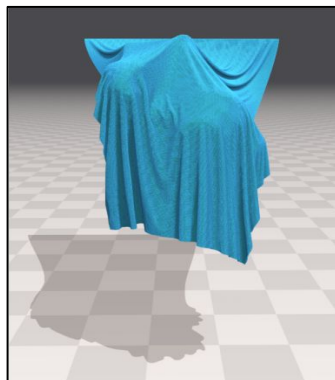
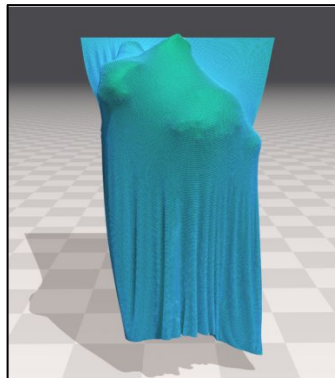
- Local Solver (no system assembly)
- Fast: GPU friendly
- Robust: analytical solve
- Simple

Small Steps (XPBD) [Macklin 2019]

- 1 iteration and multiple time steps
- Faster convergence
- Same cost

High-Order elements [Schneider 2018-2022]

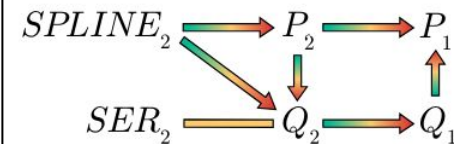
- Less topology dependent
- Better precision than linear elements
- No locking artifacts



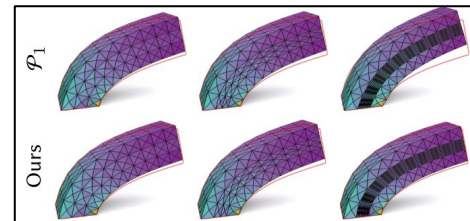
[Macklin 2019]



[Chen 2024]



[Schneider 2022]



[Schneider 2018]

[Macklin 2019] - Small Steps in Physics Simulation

[Chen 2024] - Vertex Block Descent

[Schneider 2018] - Decoupling Simulation Accuracy from Mesh Quality

[Schneider 2022] - A Large-Scale Comparison of Tetrahedral and Hexahedral Elements for Solving Elliptic PDEs with the Finite Element Method

Previous Limitations and Contributions

Advantage (VBD) :

- Fast: GPU friendly
- Robust: analytical solve
- Energy based
- Simple

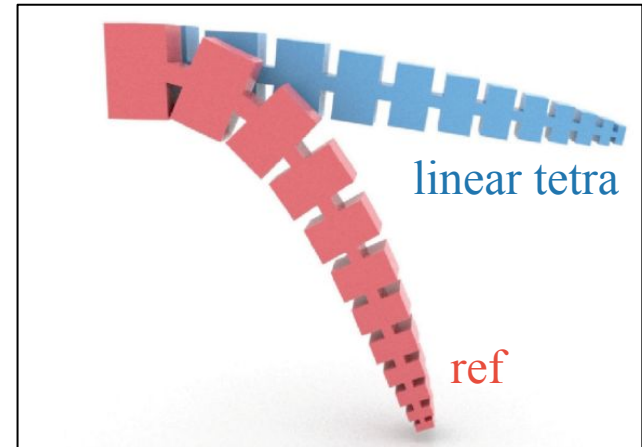
Limitations :

- FEM: Linear tetrahedron only (locking)
- Bad convergence with dense mesh ~
- Can't represent very stiff material

Contributions :

- Improved accuracy with high-order elements
- Faster High-order elements
- Better convergence with sub-steps like XPBD

Locking Artifact



[Schneider 2022] - Locking Free Tetrahedra

Summary

Problematic and Contributions

Introduction

Method

Results

Conclusion and Limitations

VBD : Local optimization integrator

Global Energy
[Gast 2014]

$$G(\mathbf{x}) = \underbrace{\frac{1}{2h^2} \|\mathbf{x} - \mathbf{y}\|_M^2}_{\text{inertia}} + \underbrace{E(\mathbf{x})}_{\text{elastic}}$$

Local energy

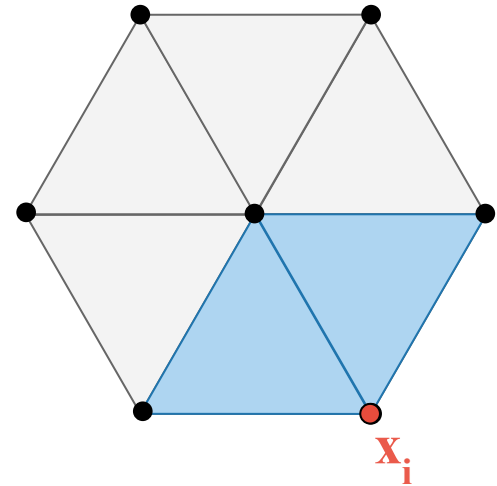
$$G_i(\mathbf{x}) = \frac{m_i}{2h^2} \|\mathbf{x}_i - \mathbf{y}_i\|^2 + \sum_{j \in \mathcal{F}_i} E_j(\mathbf{x})$$

Local Minimization

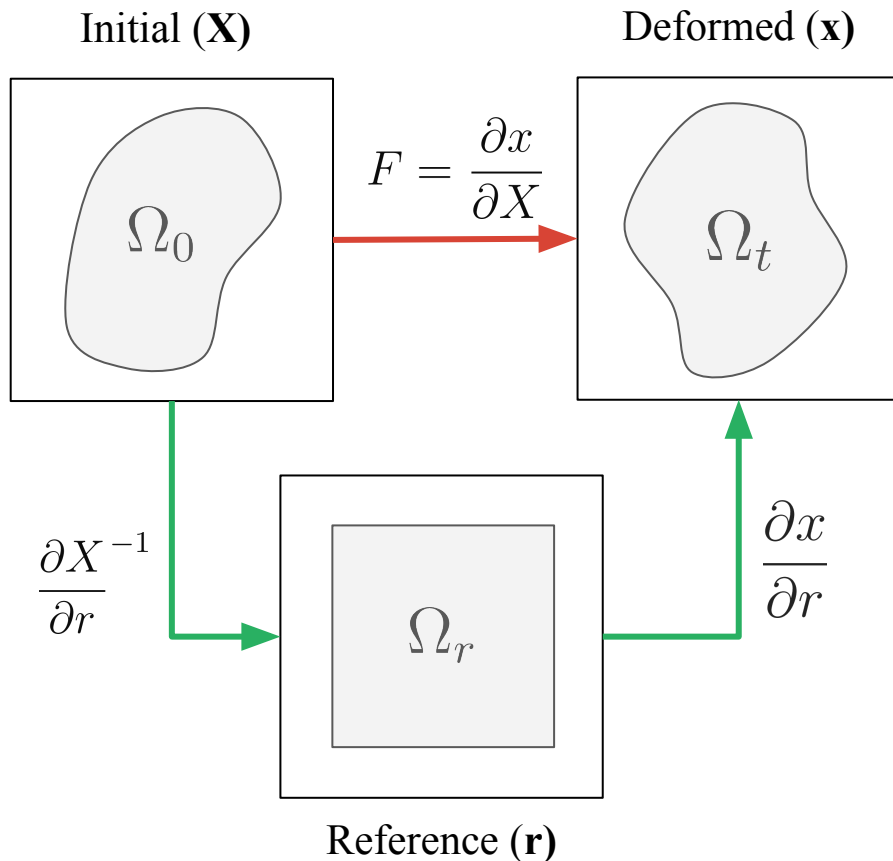
$$\mathbf{x}_i \leftarrow \underset{\mathbf{x}_i}{\operatorname{argmin}} G_i(\mathbf{x})$$

System solve

$$\Delta x_i = \underbrace{\left[\frac{\partial^2 G_i(x)}{\partial x_i^2} \right]^{-1}}_{\substack{3 \times 3 \\ \text{Hessian Matrix}}} \underbrace{\frac{\partial G_i(x)}{\partial x_i}}_{\substack{3 \times 1 \\ \text{Force Vector}}}$$



Isoparametric Elements



How to compute ?

$$E = \int_{\Omega} \Psi(F) dx$$

$$\frac{\partial E}{\partial x} = \int_{\Omega} \frac{\partial \Psi(F)}{\partial F} \frac{\partial F}{\partial x} dx$$

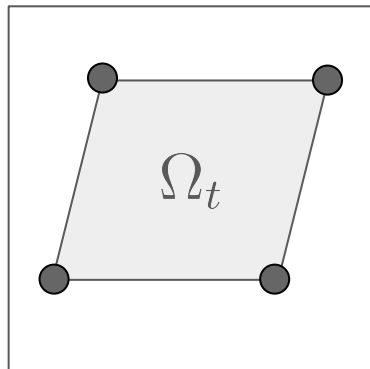
$$\frac{\partial^2 E}{\partial x^2} = \int_{\Omega} \frac{\partial F^T}{\partial x} \frac{\partial^2 \Psi(F)}{\partial F^2} \frac{\partial F}{\partial x} dx$$

Deformation gradient

$$F = \frac{\partial x}{\partial r} \frac{\partial X^{-1}}{\partial r}$$

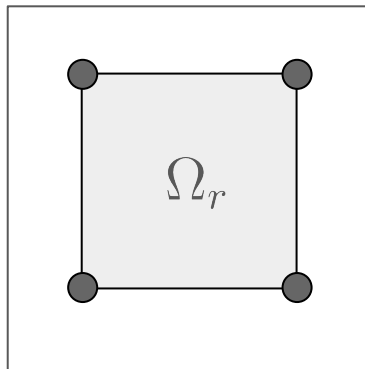
Quadratures

$$E = \int_{\Omega} \Psi(F) dx$$



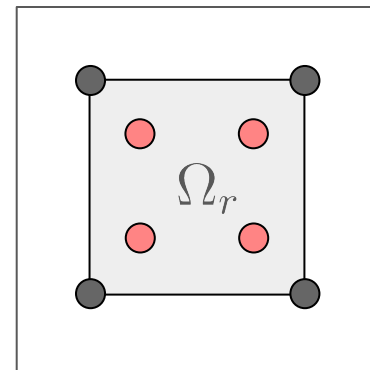
Need to evaluate integrals

$$E = \int_{\Omega_r} \Psi(F) \det(J(X)) dr$$



Evaluated in the reference element

$$E = \sum_q^Q V_q \Psi(F_q) dr$$
$$V_q = w_q \det(J_q(X))$$



Easy numerical quadratures

Contribution 1: local formulation for all iso-parametric elements [in paper]

All quadratures can be evaluated independently

Summary

Problematic and Contributions

Introduction

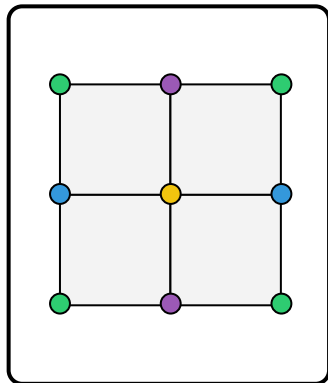
Method

Results

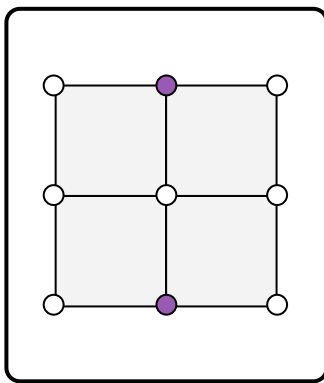
Conclusion and Limitations

VBD : Ours extension

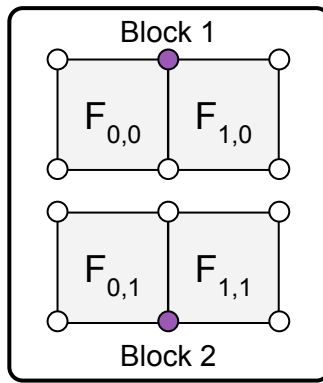
Coloration [init]



1 - Take 1 color



2 - Create Blocks



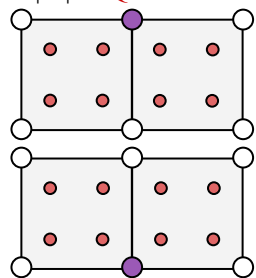
Contribution 2
Efficient parallelized quadratures

$$G_i(\mathbf{x}) = \frac{m_i}{2h^2} \|\mathbf{x}_i - \mathbf{y}_i\|^2 + \sum_{j \in \mathcal{F}_i} E_j(\mathbf{x})$$



$$G_i(x) = \frac{m_i}{2h^2} \|x_i - y_i\|^2 + \sum_{j \in \mathcal{F}_i} \sum_q^Q E_{j,q}(x)$$

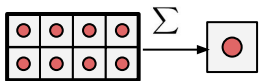
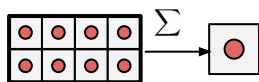
$|\mathcal{F}_i| \times Q$ threads



$|\mathcal{F}_i| \times Q$ threads

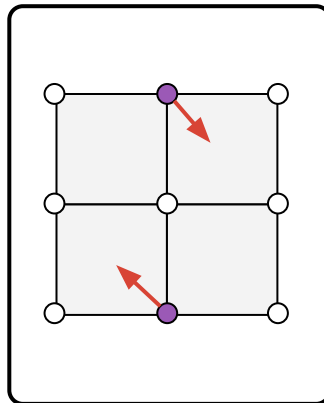
3 - Create Threads

Shared 1



Shared 2

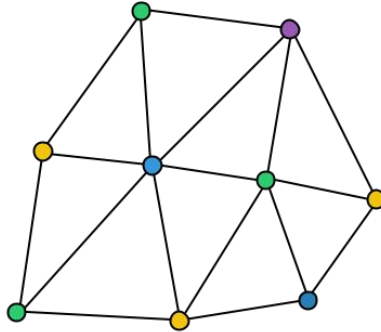
4 - Store & Reduction



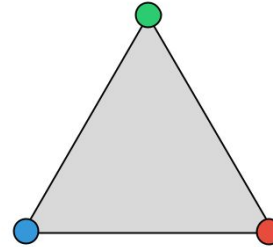
5 - Correction

Coloration limitation with high-order

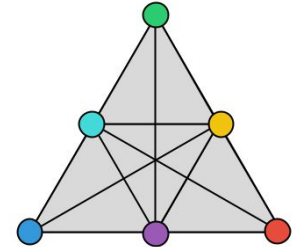
Primal / Topology



Linear



Quadratic



But

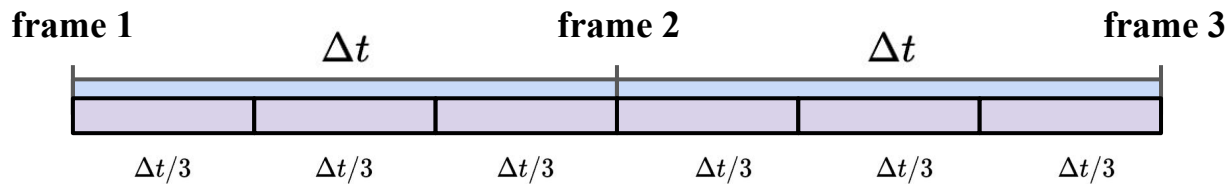
VBD color directly the primal graph

Permit efficient coloration !

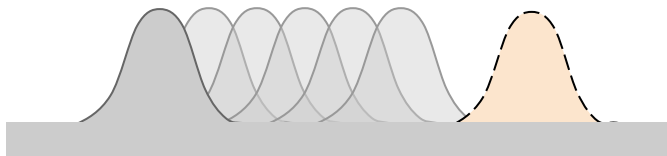
All element's vertices are linked

Chromatic number is bounded by elements

Sub-steps in VBD



Large time steps



Current solution is far from optimum

VBD solve error in close neighbors at each iteration

Hard to reduce global error

Small time steps



Current solution is close to optimum (local error)

Local optimization becomes more effective

Substeps

Multiple temporal integrations between two frames

Summary

Problematic and Contributions

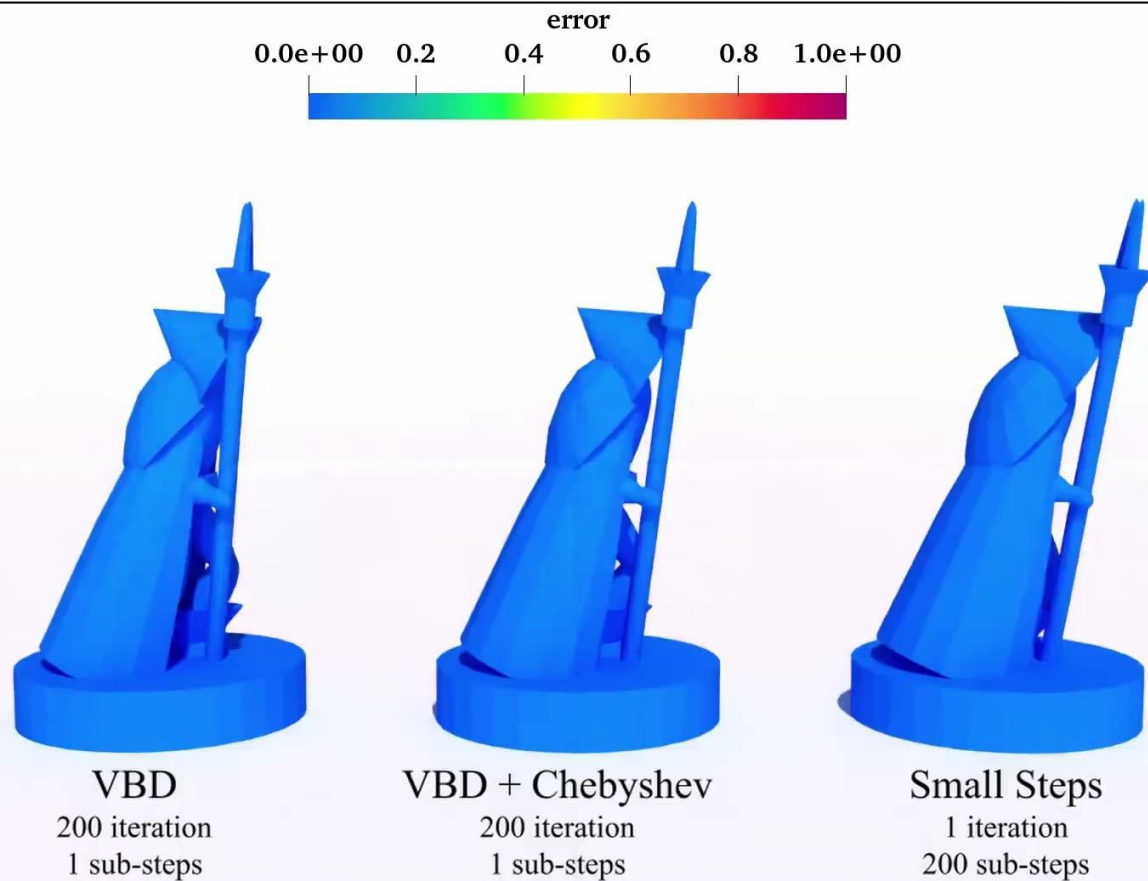
Introduction

Method

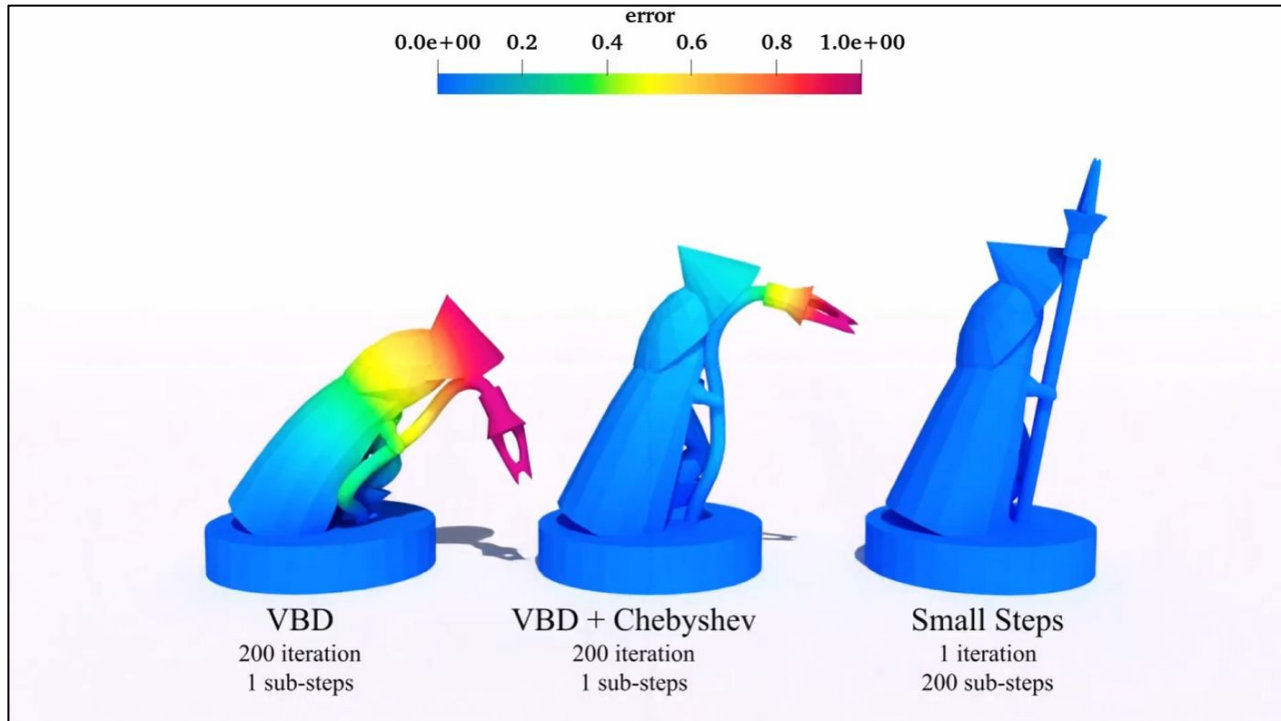
Results

Conclusion and Limitations

Sub-steps *VS* Chebyshev



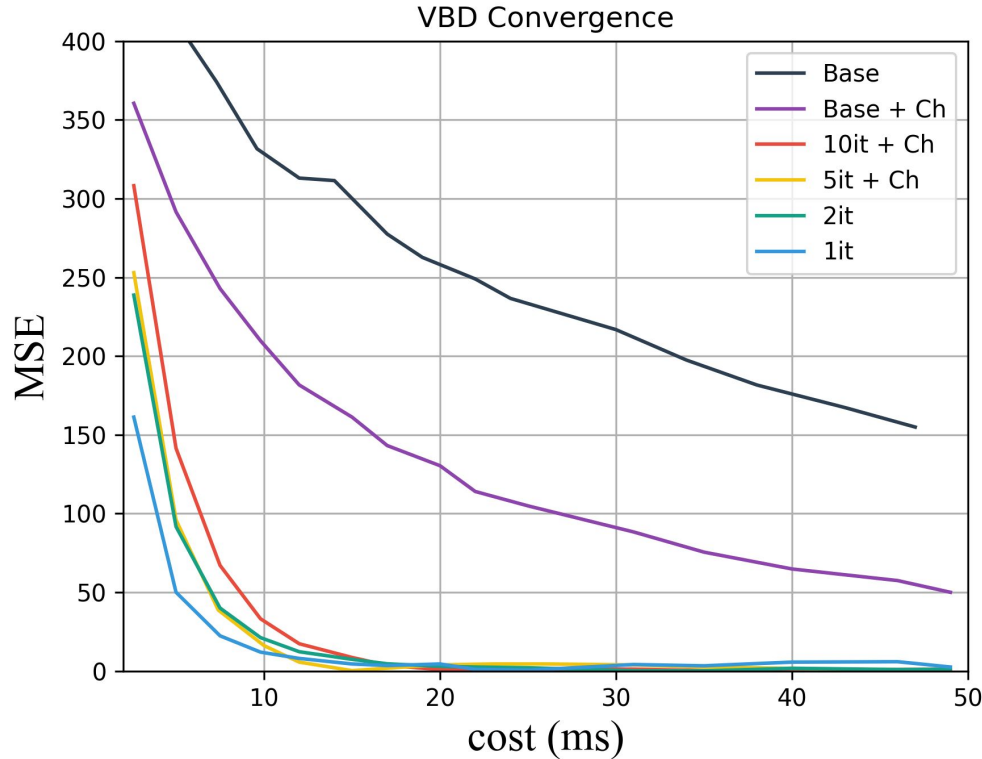
Small Steps *VS* Chebyshev



VBD use Chebyshev to increase convergence

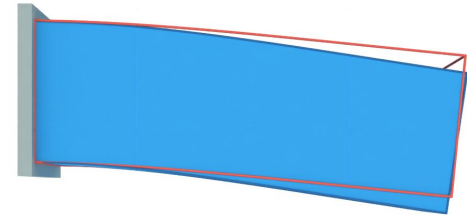
Sub-steps achieve better result without it

Small Steps: Convergence (P1)



Base = Baseline VBD with increasing iterations
#it = number of fixed iteration
Ch = With Chebyshev acceleration
MSE = Mean distance between current and converged solution

Bending test
(red = converged solution)



Fast error reduction with Sub-steps

Number of iterations depends on the scene

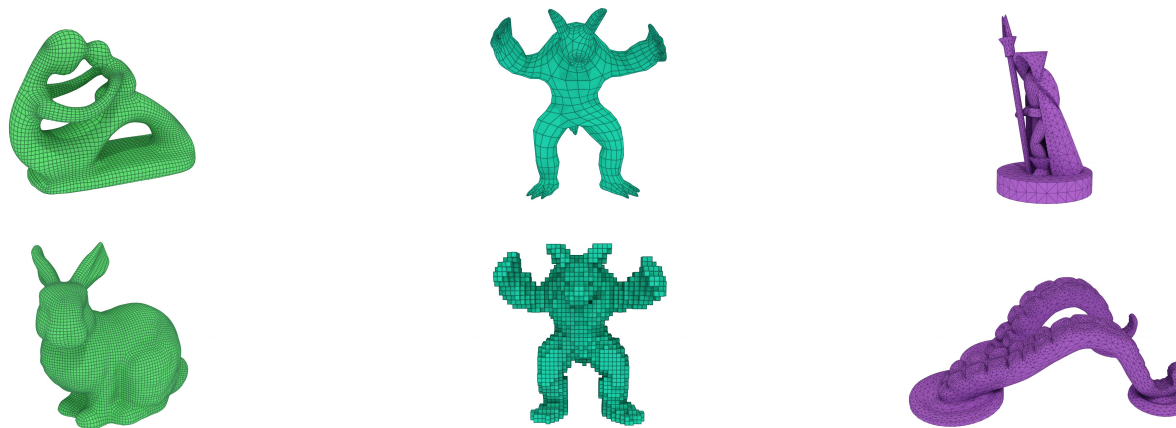
Significant speed up

Contribution 3
Small Steps relevance in VBD

Performance: parallelized quadratures

	Element	Model	#vertices	#elements	Base (ms)	Ours (ms)	#Colors	Gain
Linear hexaedra	Q_1	Fertility	20,700	24,468	72.9	44.1	13	1.65
		Stanford bunny	65,603	59,841	173.3	95.9	14	1.81
Quadratic hexaedra	Q_2	Armadillo (low poly)	35,601	3,856	641	253	30	2.53
		Armadillo (voxel)	68,165	7,229	1,113	323	28	3.45
Quadratic tetraedra	P_2	Miniature	34,023	20,253	100.9	71.8	14	1.41
		Tentacle Miniature	80,157	46,944	222	147.5	14	1.51

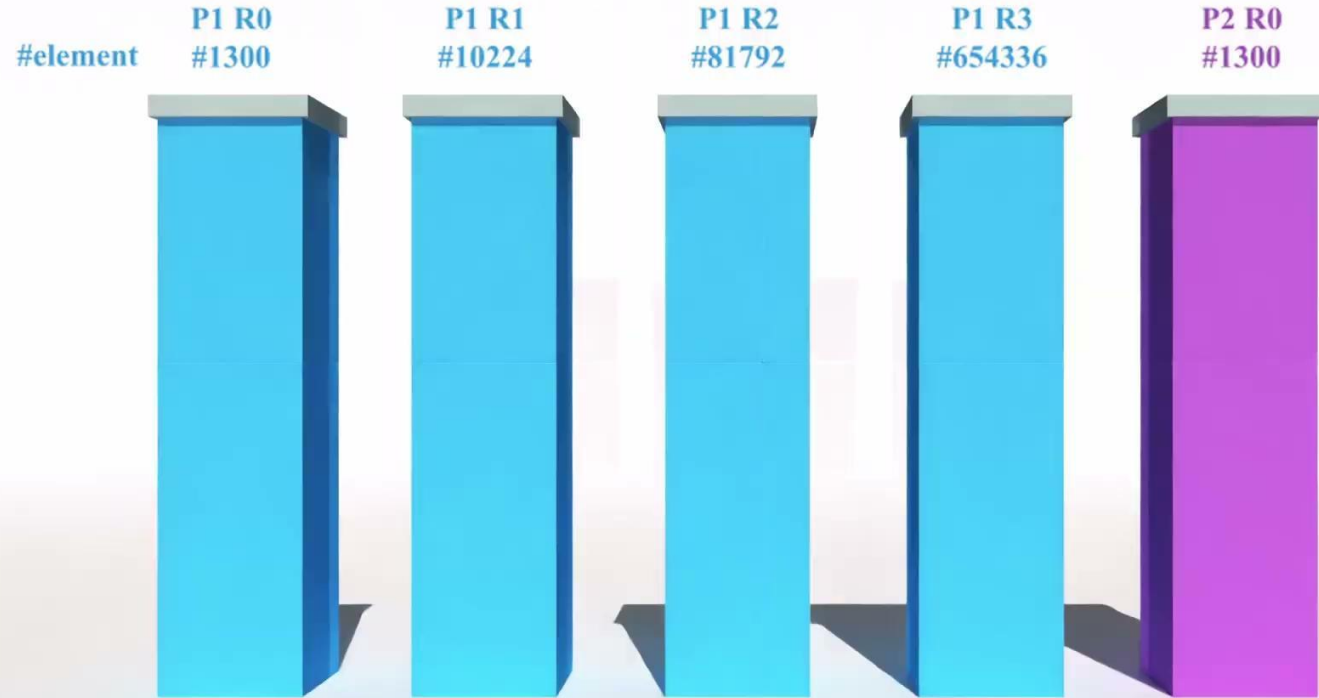
Baseline
Ours



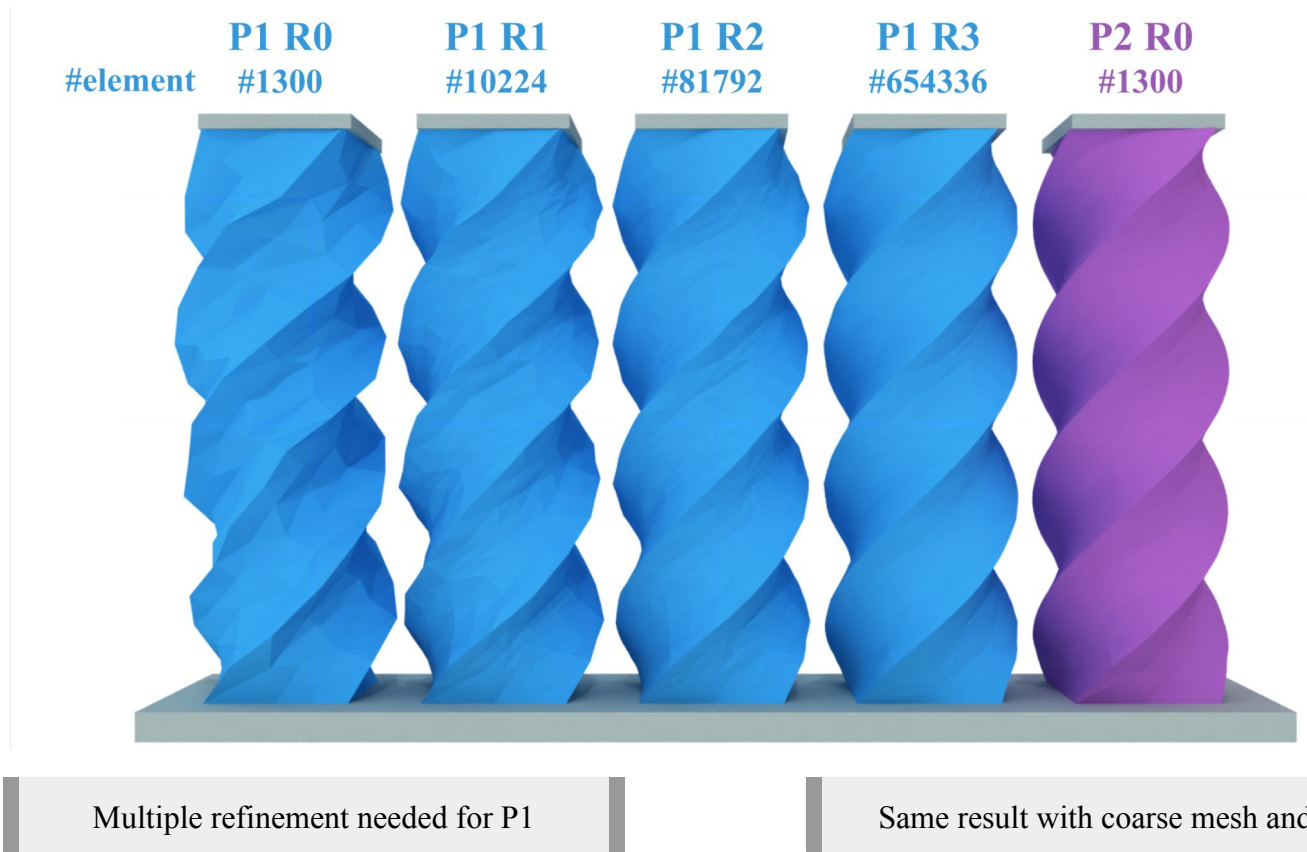
Substantial speed up for element other than P1

Scales even better with increasing quadrature

Accuracy: Visual Comparison



Accuracy: Visual Comparison



R# = number of mesh refinement

P1 = linear tetrahedra

P2 = Quadratic tetrahedra

Accuracy: Elements comparison

Beam	#vertices	#elem	t (ms)	e (m)
P_1	2,417	10,240	7.5	$1.0e^{-1}$
Q_1	2,673	2,048	7.5	$7.4e^{-2}$
P_2	425	160	6.6	$2.5e^{-2}$
Q_2	81	4	8.3	$2.0e^{-1}$

P_1	16,225	81,920	20.2	$2.93e^{-2}$
Q_1	18,785	16,384	19.3	$7.7e^{-3}$
P_2	11,453	6,860	18	$9.0e^{-5}$
Q_2	1,225	108	18.3	$4.5e^{-3}$

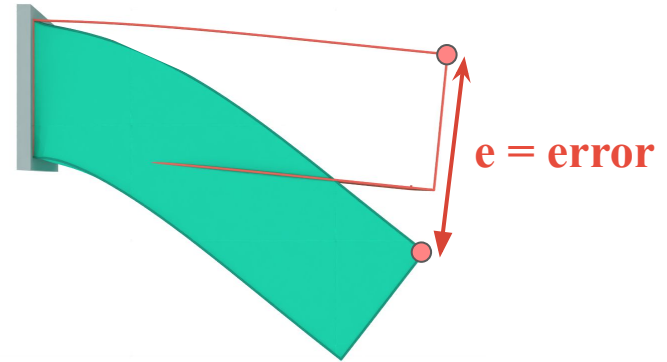
P_1	30,721	160,000	31.7	$1.26e^{-2}$
Q_1	26,353	23,328	27	$3.96e^{-3}$
P_2	16,225	10,240	32	$3.80e^{-6}$
Q_2	4,961	500	41	$8.13e^{-4}$

Q1 = Linear hexahedra

Q2 = Quadratic hexahedra

P1 = Linear tetrahedra

P2 = Quadratic tetrahedra

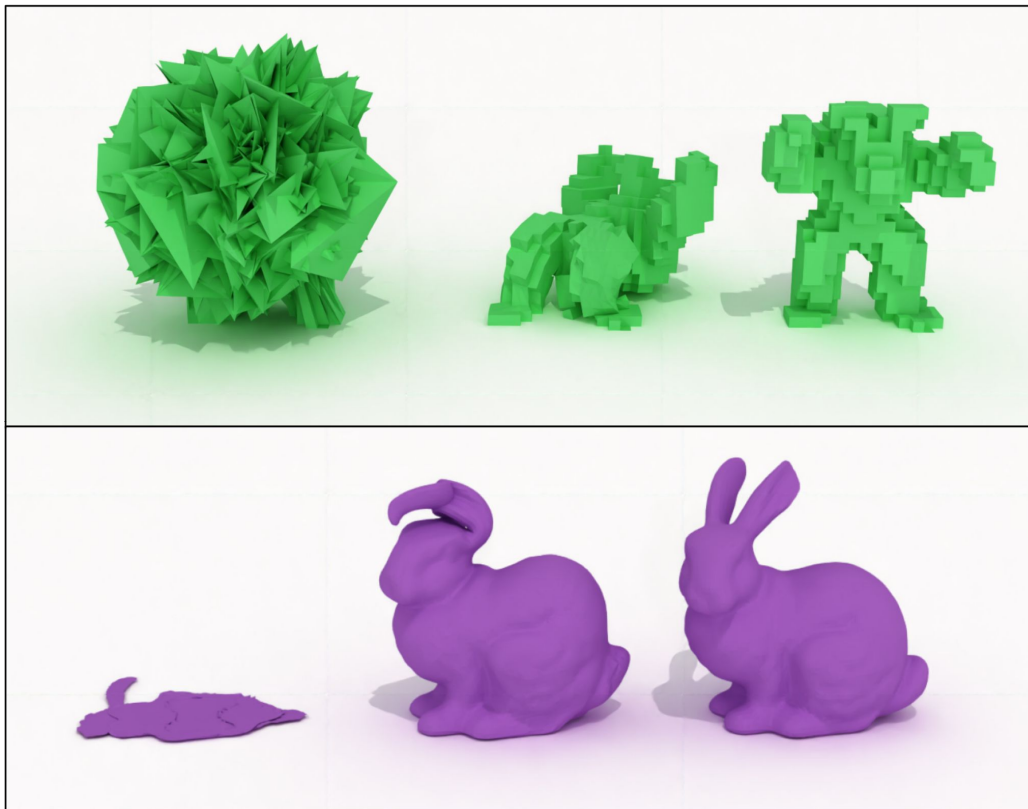


All elements scale better than P1

P2 are the best elements here

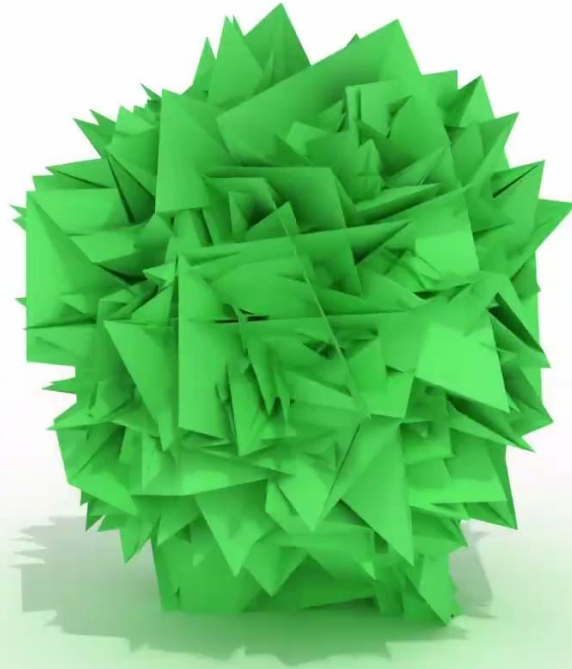
Q2 have too much vertices per element

Robust: Highly inverted mesh



Goal: Test Q1 robustness with inversions

Robust: Highly inverted mesh



Robust: Hard Twist

P1



P2



Q1



Q2



Goal: Test all elements robustness with large deformation

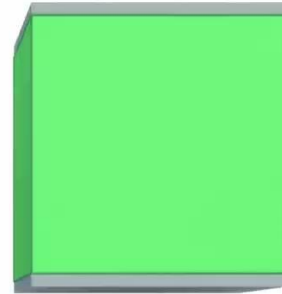
Test used by Macklin et al. and Smith et al.

Robust: Hard Twist

P1



Q1



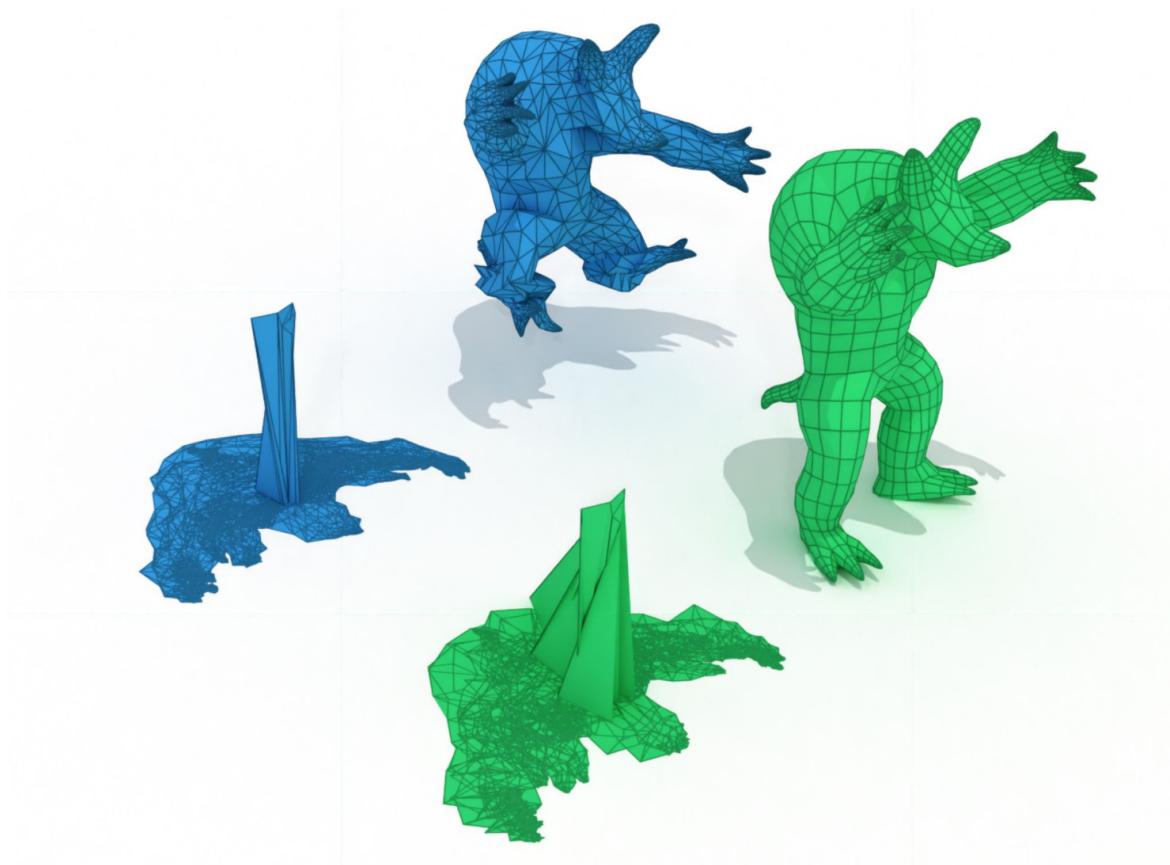
P2



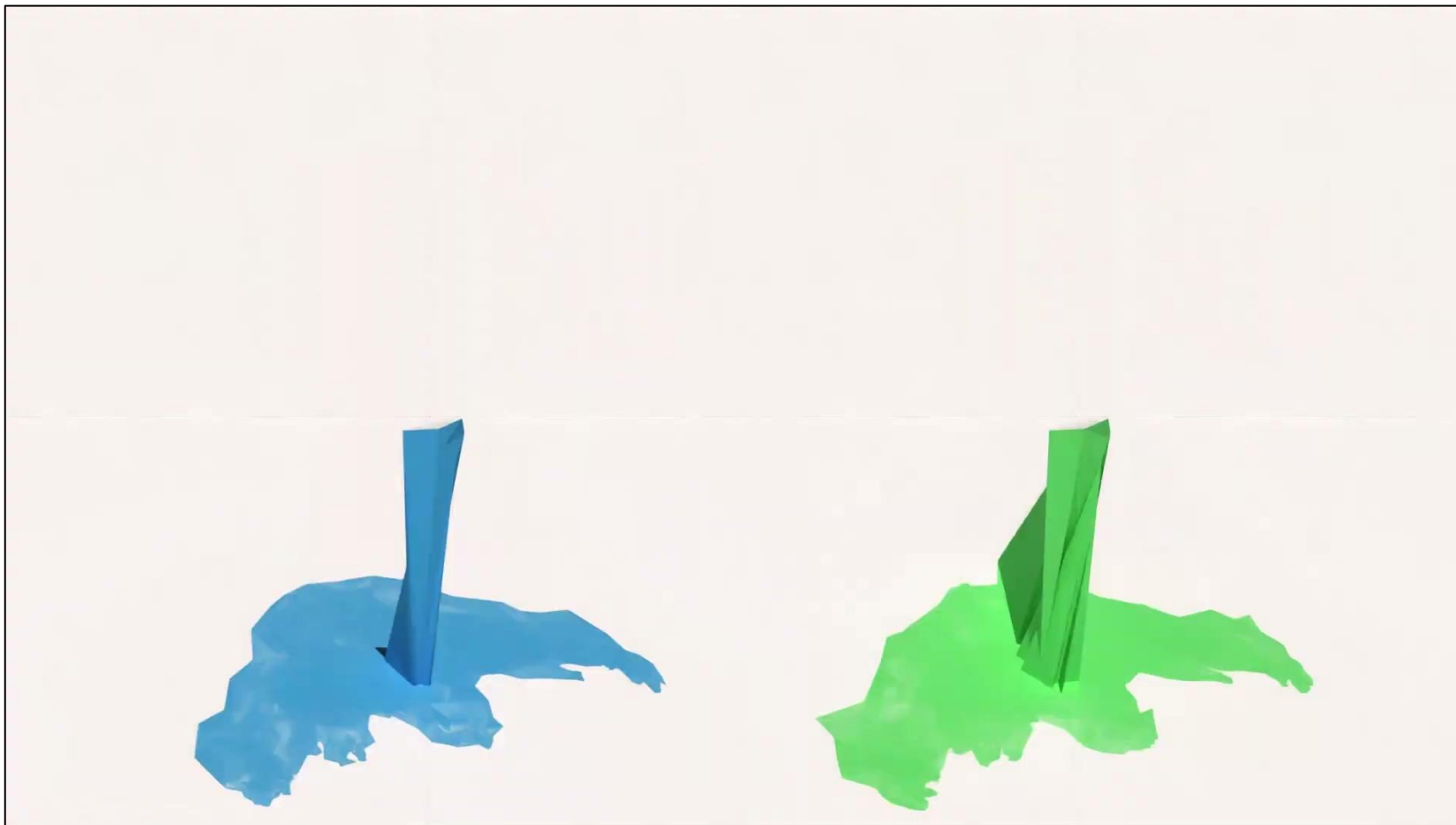
Q2



Robust: Inversion Recovery



Robust: Inversion Recovery



Summary

Problematic and Contributions

Introduction

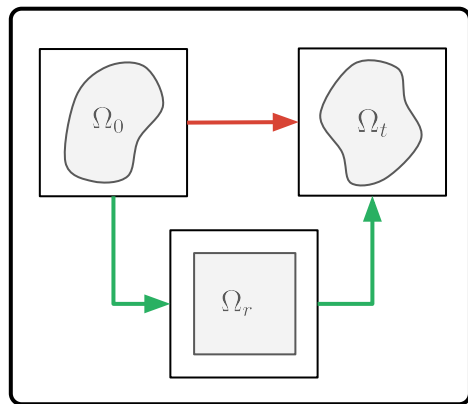
Method

Results

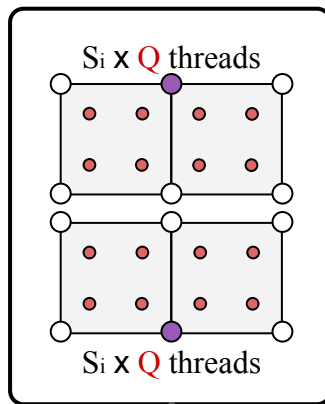
Conclusion and Limitations

Summary

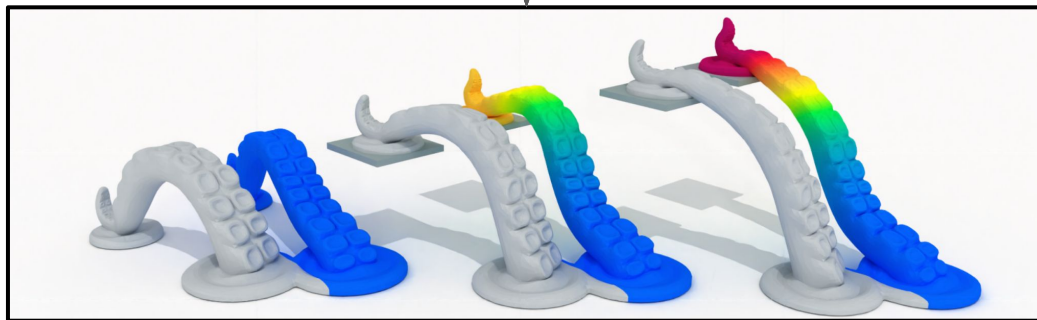
1- Generic Formulation



2- Parallelized Quadrature



3- VBD small steps



Conclusion and Limitations

Faster High-order within VBD

Precision: $P2 > Q2 > Q1 > P1$

P2 and Q2 Can represent curved surfaces

Still very stable

Inversion recovery: $Q1 > P1$

High-order element are sensible to inversion

Bad convergence with dense volume

Can't perfectly preserve volume

Perspectives

Locking Free VBD with Linear Tetraedra

Multi-Scale VBD

Thank You!

Contact: bastien.saillant@liris.cnrs.fr

